

基于 PagedAttention 的大语言模型高效内存管理

Woosuk Kwon*, Zhuohan Li* 等 | UC Berkeley / Stanford / UCSD | SOSP'23
arXiv:2309.06180v1 | 严格对照原文翻译 (v2)

摘要 (Abstract)

大语言模型 (LLM) 的高吞吐服务需要同时批处理足够多的请求。然而，现有系统面临困难，因为每个请求的键值缓存 (KV cache) 内存占用巨大，且随时间动态增减。当管理不善时，该内存会因碎片化 (fragmentation) 和冗余复制 (redundant duplication) 而被严重浪费，从而限制批处理大小。

为解决此问题，我们提出 PagedAttention——一种受操作系统经典虚拟内存与分页技术启发的注意力算法。在此基础上，我们构建了 vLLM，一个 LLM 服务系统，实现了：(1) KV cache 内存接近零浪费；(2) 在请求内部和请求之间灵活共享 KV cache，进一步减少内存使用。

评估表明，vLLM 在相同延迟水平下将主流 LLM 的吞吐量提升 2-4 倍 (对比 FasterTransformer 和 Orca)。序列越长、模型越大、解码算法越复杂时改进越显著。

[原文] High throughput serving of LLMs requires batching sufficiently many requests at a time. However, existing systems struggle because the key-value cache (KV cache) memory for each request is huge and grows and shrinks dynamically. When managed inefficiently, this memory can be significantly wasted by fragmentation and redundant duplication, limiting the batch size.

1. 引言 (Introduction)

GPT、PaLM 等大语言模型催生了编程助手、通用聊天机器人等新应用。云厂商争相提供托管服务，但运行成本极高——处理一次 LLM 请求的开销是传统关键词查询的 10 倍。因此提升 LLM 服务系统的吞吐量 (降低单请求成本) 变得至关重要。

LLM 的核心是自回归 Transformer 模型，每次生成一个 token，重复直至终止符。这种顺序生成过程使工作负载是内存瓶颈型 (memory-bound)，GPU 算力利用不足。可通过批处理多请求来提升吞吐，但前提是每个请求的内存空间须高效管理。

以 13B 模型在 A100 40GB 为例 (Fig.1 左)：模型权重约 65% (26GB)，KV cache 动态占用 >30%，剩余少量用于激活值。KV cache 管理效率直接决定最大批处理大小。

现有系统的两个核心缺陷：

缺陷一：内部与外部碎片。为将请求的 KV cache 存储在连续空间中，现有系统按请求的最大长度 (如 2048 token) 预分配连续内存块。这导致严重的内部碎片 (请求实际长度远小于最大长度)，同时不同请求的预分配大小不同，释放后产生外部碎片。实测 (Fig.2) 仅 20.4%-38.2% 的 KV cache 内存用于存储实际 token 状态。

[原文] First, the existing systems suffer from internal and external memory fragmentation. To store the KV cache of a request in contiguous space, they pre-allocate a contiguous chunk of memory with the request's maximum length. This can result in severe internal fragmentation... Indeed, our profiling results in Fig. 2 show that only 20.4% - 38.2% of the KV cache memory is used to store the actual token states in the existing systems.

缺陷二：无法利用内存共享。并行采样、Beam Search 等高级解码算法为一个请求生成多个输出序列，这些序列可部分共享 KV cache。但现有系统因 KV cache 存储在独立连续空间中，无法实现内存共享。

我们的贡献：

- 识别 LLM 服务中内存分配的挑战并量化其对性能的影响
- 提出 PagedAttention——在非连续分页内存上运行的注意力算法 (受 OS 虚拟内存/分页启发)
- 设计并实现 vLLM——基于 PagedAttention 的高吞吐分布式 LLM 服务引擎
- 在多种场景下展示 vLLM 大幅超越 FasterTransformer 和 Orca

2. 背景 (Background)

2.1 基于 Transformer 的大语言模型

语言模型的任务是对 token 序列建模概率分布。当前最成功的方法是 Transformer，核心组件是自注意力层：输入序列的每个 token 映射为 Query(Q)、Key(K)、Value(V) 向量，注意力输出为 $\text{softmax}(\text{QK}^T / \sqrt{d_{\text{head}}}) \cdot V$ 。

LLM 的生成过程分两个阶段：

(1) Prompt 阶段 (Prefill)：将用户完整 prompt ($x_1, \dots, x_n$) 一次性输入，计算第一个新 token 的概率；这些 prompt 的 KV 向量被缓存下来。

(2) 自回归生成阶段 (Decode)：每次只输入上一步生成的 token，计算新的 KV 并拼接到历史 KV 后做 attention。为避免重复计算历史 token 的 KV，所有已计算的 Key 和 Value 被缓存——即 KV cache。

2.2 LLM 服务与迭代级调度 (Iteration-Level Scheduling)

传统请求级批处理 (一批全完成才开始下一批) 效率低。Orca [60] 提出迭代级调度：每完成一个 decode step 就重新决定 batch 组成——已完成的请求移出、新请求加入。这为动态批处理奠定基础。

3. LLM 服务中的内存管理挑战 (Memory Challenges in LLM Serving)

即使有迭代级调度，能批处理的请求数仍受 GPU 内存容量限制——特别是分配给 KV cache 的空间。这是内存瓶颈 (memory-bound) 的本质。克服此瓶颈需要解决以下挑战：

大型 KV cache (Large KV cache)

KV cache 随请求数量快速增长。以 OPT-13B 为例，单个 token 的 KV cache 需 800KB (计算： $2(K+V) \times 5120(\text{hidden}) \times 40(\text{层}) \times 2(\text{FP16}) = 800\text{KB}$)。一个请求最多 2048 token，KV cache 可达 1.6GB。当前 GPU 内存数十 GB，即使全分配给 KV cache 也只能容纳几十个请求。且内存管理低效会进一步减少批处理大小 (如 Fig.2 所示)。

[原文] The KV Cache size grows quickly with the number of requests. As an example, for the 13B parameter OPT model, the KV cache of a single token demands 800 KB of space, calculated as $2 \times (\text{key and value vectors}) \times 5120 (\text{hidden state size}) \times 40 (\text{number of layers}) \times 2 (\text{bytes per FP16})$.

【译注】这里原文给出的 OPT-13B 公式是： $2(K+V) \times \text{hidden_size} \times \text{num_layers} \times \text{bytes_per_element}$ 。如果用 multi-head 拆分表达： $2 \times \text{num_layers} \times \text{num_heads} \times \text{head_dim} \times \text{bytes}$ 。两种写法等价 (因为 $\text{hidden_size} = \text{num_heads} \times \text{head_dim}$)。

复杂解码算法 (Complex decoding algorithms)

LLM 服务提供多种解码方法 (贪心、并行采样、Beam Search 等)，各有不同内存管理复杂度。并行采样时 prompt 部分 KV 可共享 (占总 KV 约 12%)，Beam Search 中不同 beam 可共享更大比例 (多达 55%)。共享程度取决于具体解码算法，且共享模式随解码推进动态变化。

输入输出长度未知 (Unknown input & output lengths)

请求的输入和输出长度各异且事先未知。内存管理系统需适应广泛的 prompt 长度，且随着输出增长 KV cache 不断扩大，可能耗尽可用内存。系统需做调度决策——如删除或交换某些请求的 KV cache。

3.1 现有系统的内存管理 (Memory Management in Existing Systems)

由于深度学习框架要求 tensor 存储在连续内存中，现有系统 [31,60] 将请求的 KV cache 作为连续 tensor 存储。由于输出长度不可预测，系统按请求的最大可能序列长度静态分配内存。

Fig.3 (第4页图) 展示了两个请求的例子：Request A 最大长度 2048，Request B 最大长度 512。这种预分配方案有三种内存浪费来源：

- ① Reserved slots (预留槽位)：为未来可能生成的 token 预留的空间。虽然最终会被使用，但在请求整个生命周期内，这些空间被独占、不能被其他请求利用。
- ② Internal fragmentation (内部碎片)：请求实际生成的 token 数少于最大长度时，剩余的预分配空间永远不会被使用。只有在请求完成采样后才知道浪费了多少。
- ③ External fragmentation (外部碎片)：不同请求的预分配块大小不同，释放后产生大小不一的空闲间隙，导致新请求即使总空闲内存充足也可能找不到足够大的连续空间。

[原文] The chunk pre-allocation scheme in existing systems has three primary sources of memory wastes: reserved slots for future tokens, internal fragmentation due to over-provisioning for potential maximum sequence lengths, and external fragmentation from the memory allocator like the buddy allocator.

Fig.2 量化了这些浪费：Orca(Max) 有效利用率仅 20.4%，Orca(Pow2) 为 26.8%，即使理想化的 Orca(Oracle) (事先知道精确长度) 也只有 57.3% (仍有外部碎片)。vLLM 达到 96.3%。

4. 方法 (Method)

我们开发了一种新的注意力算法 PagedAttention，并在此基础上构建 LLM 服务引擎 vLLM。vLLM 的架构 (Fig.4) 包含：一个中心化调度器 (scheduler) 协调分布式 GPU worker 的执行；一个 KV cache 管理器 (KV cache manager) 以分页方式管理 KV cache。

4.1 PagedAttention 算法

为解决 §3 的内存挑战，我们引入 PagedAttention——受操作系统中分页 (paging) 经典思想启发的注意力算法。与传统 attention 算法不同，PagedAttention 允许将 Key 和 Value 存储在非连续的内存空间中。

具体而言，PagedAttention 将每个序列的 KV cache 划分为 KV blocks。每个 block 包含固定数量 token (记为 block size B) 的 Key 和 Value 向量。

[原文] Specifically, PagedAttention partitions the KV cache of each sequence into KV blocks, each of which can contain the attention keys and values of a fixed number of tokens.

记第 j 个 key block 为 $K_j = (k_{(j-1)B+1}, \dots, k_{jB})$ ，value block 类似。attention 计算可转化为按 block 进行：

$$A_{ij} = \exp(q_i^T \cdot K_j / \sqrt{d}) / \sum_l \exp(q_i^T \cdot K_l / \sqrt{d}) \quad (\text{公式 4})$$
$$o_i = \sum_j V_j \cdot A_{ij}^T$$

其中 A_{ij} 是 query i 对第 j 个 KV block 的注意力分数向量。关键点：PagedAttention kernel 逐 block 获取 K/V 数据，分别计算 raw score，最后做全局 softmax——数学结果与标准 attention 完全等价。

Fig.5 展示了 PagedAttention 的示例：query token "forth" 与三个非连续物理 block 中的 Key 做点积。三个 block 在物理内存中不必相邻。

4.2 KV Cache 管理器 (KV Cache Manager)

vLLM 的内存管理核心思想类似于操作系统的虚拟内存：

- OS 将内存分为固定大小的页 (pages)，通过页表 (page table) 映射逻辑地址到物理地址
- vLLM 将 KV cache 分为固定大小的 KV block，通过 block table 映射逻辑 block 到物理 block
- 物理内存不需预先全部分配，OS 按需分配物理页 → vLLM 按需分配物理 block

一个请求的 KV cache 表示为一系列逻辑 KV block，从左到右填充。GPU worker 上，block engine 预分配一块连续 GPU DRAM，划分为物理 KV block。Block table 记录逻辑 block → 物理 block 的映射关系以及每个 block 已填充的 token 数 (#filled)。

[原文] A request's KV cache is represented as a series of logical KV blocks, filled from left to right as new tokens and their KV cache are generated. The last KV block's unfilled positions are reserved for future generations. On GPU workers, a block engine allocates a contiguous chunk of GPU DRAM and divides it into physical KV blocks.

Fig.6 展示了 block table 的翻译过程：逻辑 block 0/1/2 分别映射到物理 block 7/1/3。分离逻辑和物理 block 使 vLLM 无需为所有位置预留内存，消除了大部分内存浪费。

Fig.7 展示了两个请求同时服务时的内存布局：两个请求的物理 block 在 GPU DRAM 中交错分布，彼此不需要连续空间。

4.3 解码过程与 PagedAttention 配合 (Decoding with PagedAttention)

以一个 prompt 为 7 个 token 的请求为例 (Fig.6 左侧)：

① 初始分配：vLLM 将前 4 个 token 的 KV 放入逻辑 block 0 (映射到物理 block 7)，后 3 个 token 放入逻辑 block 1 (映射到物理 block 1)，block 1 还剩 1 个空位。

- ② 第一步生成：新 token 的 KV 填入逻辑 block 1 的最后一个空位。Block 1 满了。
- ③ 第二步生成：逻辑 block 1 已满，vLLM 从 block pool 分配新的物理 block (block 3)，建立逻辑 block 2 → 物理 block 3 的映射，新 token KV 写入。
- ④ 请求结束：所有物理 block 归还到全局 pool。

全局看：每个 decode 迭代，调度器选一组请求进入 batch，为新 token 分配物理 block，将 input token ID 和 block table 发给 GPU worker，worker 用 PagedAttention kernel 执行 attention (从 block table 读取物理地址获取 K/V)。

vLLM 将所有内存浪费限制在每个请求最后一个 block 的未填满部分 (最多浪费 $\text{block_size} - 1$ 个 token 的空间)。这使得 vLLM 能有效利用几乎所有内存用于批处理，大幅提升吞吐量。

4.4 应用于其他解码场景 (Application to Other Decoding Scenarios)

并行采样 (Parallel Sampling)

同一 prompt 生成多个输出样本时 (如 $n=2$)，所有样本共享 prompt 阶段的 KV block，通过 reference count (引用计数) 管理共享 block。Fig.8 展示了两个样本共享 prompt 的物理 block (block 7 和 1 的 $\text{ref_count}=2$)。

生成阶段各样本独立写新 block。当某样本要写入一个 $\text{ref_count} > 1$ 的 block 时，触发 copy-on-write (CoW)：复制该 block 到新物理 block，然后写入。

[原文] Since both outputs share the same prompt, we only reserve space for one copy of the prompt's state at the prompt phase... vLLM implements a copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences.

Beam Search

Beam Search 中多个候选序列 (beam) 共享不同长度的公共前缀，且共享模式随解码动态变化 (类似 OS 中 `fork()` 创建的进程树)。Fig.9 展示了 $\text{beam width}=4$ 的例子：

- 所有 beam 共享 block 0 (prompt)
- Candidate 0-2 共享前 3 个 block，Candidate 3 在第 2 个 block 分叉
- 某些 candidate 被淘汰 → 对应 block 的 ref_count 减少 → 到 0 时释放
- 新 beam 分配新 block 存储各自生成的 KV

Copy-on-Write 只在实际需要修改共享 block 时触发，大幅减少了传统实现中的完整 KV 复制。

共享前缀 (Shared Prefix / System Prompt)

常见场景：多用户共享相同的 system prompt (如翻译任务的 few-shot 示例，Fig.10)。vLLM 预先为共享前缀分配一组物理 block，新请求直接将逻辑 block 映射到这些已有物理 block (最后一个 block 标记为 copy-on-write)。只需对用户实际输入部分执行 prompt 计算。类似于 OS 中进程间共享动态库 (shared library) 的机制。

4.5 调度与抢占 (Scheduling and Preemption)

当请求流量超过系统容量时，vLLM 需要对请求进行优先级排序。采用先来先服务 (FCFS) 调度策略，确保公平性、防止饥饿。抢占时优先抢占最后到达的请求 (确保先到的请求优先完成)。

LLM 服务面临独特挑战：输入长度各异、输出长度事先未知。随着请求数增长和输出扩展，vLLM 可能耗尽物理 block。此时需要回答两个经典问题：(1) 应该驱逐哪些 block？(2) 如何恢复被驱逐的 block？

对于 (1)，vLLM 采用全有或全无 (all-or-nothing) 策略——要么驱逐一个请求的全部 block，要么不驱逐。同一请求的多个序列 (如 beam search 的多个 candidate) 作为 sequence group 统一处理。

对于 (2)，vLLM 提供两种恢复策略：

① 交换 (Swapping)：将被驱逐请求的 KV block 从 GPU 复制到 CPU 内存。GPU 空闲后再换回。vLLM 包含一个 CPU block allocator 管理换出的 block。CPU 端 swap 空间大小不超过 GPU 端 KV cache 总量。适合：CPU 内存充裕、KV cache 体量中等的场景。

② 重计算 (Recomputation)：直接丢弃被驱逐请求的 KV cache。等有资源后重新执行 prompt 阶段 (将原始 prompt + 已生成的 token 合并作为新 prompt 重新计算 KV)。重计算延迟显著低于原始延迟 (因为所有位置可在一个 prompt 阶段并行计算)。适合：prompt 较短 (重算代价低)、CPU 内存紧张的场景。

[原文] Swapping: This is the classic technique used by most virtual memory implementations which copy the evicted pages to a swap space on the disk. In our case, we copy evicted blocks to the CPU memory. Recomputation: In this case, we simply recompute the KV cache when the preempted sequences are rescheduled.

4.6 分布式执行 (Distributed Execution)

很多 LLM 的参数超过单 GPU 容量，需要分布式执行。vLLM 支持 Megatron-LM 风格的张量并行 (tensor parallelism)，遵循 SPMD (单程序多数据) 执行策略。注意力层按 head 维度切分，每个 GPU 处理一部分 head。每个 GPU worker 存储对应部分 KV cache，block table 在所有 worker 间相同。调度器广播 block table 和 token ID 给所有 worker，worker 间通过 all-reduce 同步中间结果。

5. 实现 (Implementation)

vLLM 是端到端 LLM 服务系统：FastAPI 前端 + GPU 推理引擎。前端扩展 OpenAI API 接口。引擎由约 8.5K 行 Python + 2K 行 C++/CUDA 代码组成。控制面 (scheduler、block manager) 用 Python 开发；数据面 (PagedAttention kernel 等) 用自定义 CUDA kernel 开发。支持模型：GPT-2、GPT-J、OPT (125M-175B)、LLaMA (7B-65B) 等。

5.1 Kernel 级优化

PagedAttention 引入了非连续内存访问模式，需要专用 GPU kernel。三个关键优化：

- (1) Fused reshape and block write：每层 Transformer 产生新 KV 时，先 reshape 为 block 友好的内存布局，再按 block table 写入对应位置。融合为一个 kernel。
- (2) Fused block read and attention：按 block table 读 KV 并执行 attention——一个 GPU warp 负责一个 block 的读取和计算。支持 batch 内不同请求有不同序列长度。
- (3) Fused block copy：CoW 触发的 block copy 操作，将不同 block 的 copy 操作批量融合到一个 kernel launch 中，减少 launch 开销。

6. 实验评估 (Evaluation)

6.1 实验设置

- 模型：OPT-13B、OPT-175B、LLaMA-13B
- 硬件：NVIDIA A100 80GB (13B/66B 用 1 卡/4 卡，175B 用 8 卡)
- 基线：FasterTransformer [31] (NVIDIA 官方)、Orca [60] (迭代级调度最先进系统)
- 工作负载：ShareGPT (真实对话，长输出，均值 161+338 token) 和 Alpaca (短输出，均值 19+58 token)
- 指标：归一化延迟 (normalized latency) 与吞吐量 (requests/s)

6.2 主要结果

吞吐量提升 (核心数据)：

- vs FasterTransformer：vLLM 吞吐量高 14-24× (FT 不支持迭代级调度，差距最大)
- vs Orca (最优配置)：vLLM 吞吐量高 2.2×-4.3×
- 长序列 (ShareGPT) 改进更显著 (4.3×)；短序列 (Alpaca) 改进较小 (2.2×)

KV cache 有效利用率：Orca(Max) = 20.4%，vLLM = 96.3%

6.3 并行采样与 Beam Search

- 并行采样：通过内存共享，vLLM 在生成多个输出时吞吐提升更明显
- Beam Search (beam width=6)：vLLM 比 Orca 吞吐高 2.3×——CoW 机制大幅减少了 beam 间的 KV 复制开销

6.4 共享前缀 (Shared Prefix)

在翻译任务 (长 system prompt = few-shot examples) 中，共享前缀的 KV cache 复用使 vLLM 的内存效率进一步提升。

7.2 Block Size 消融实验

测试 block_size = 1, 2, 4, 8, 16, 32, 64, 128 :

- 过小 (1-4) : block table 膨胀、kernel 效率低 (warp 利用率低)
- 过大 (64-128) : 内部碎片增多 (每个请求最后一个 block 浪费大)
- 最优区间 : 16-32。论文默认选择 block_size = 16。

7.3 Kernel 微基准测试

PagedAttention kernel 相比标准连续 attention kernel 的额外开销 < 5% (间接寻址代价)。在端到端场景下几乎不影响整体性能。

8. 讨论 (Discussion)

与 OS 虚拟内存的深层对应 :

- 物理页 (physical page) → 物理 KV block
- 虚拟地址空间 → 序列的逻辑 KV 位置
- 页表 (page table) → block table
- 按需分页 (demand paging) → 按需分配 KV block
- Copy-on-Write → beam search / parallel sampling 的共享机制
- 页面换出 (swap out) → KV block 从 GPU 交换到 CPU

与 FlashAttention 的关系 :

FlashAttention 优化的是单次 attention 计算的 HBM 读写 (tiling / 不保存中间结果) , 解决的是 compute 效率问题。PagedAttention 解决的是多请求 KV cache 的内存布局/管理问题。两者正交——可以、也确实一起使用 (vLLM 的 PagedAttention kernel 内部可采用 FlashAttention 风格的 tiling)。

10. 结论 (Conclusion)

本文提出 PagedAttention——将 KV cache 组织为非连续物理 block、通过 block table 间接寻址的注意力算法——以及基于此算法的 vLLM 系统。通过消除内部/外部碎片、支持 block 级内存共享 (CoW) , vLLM 在各种模型和工作负载下大幅超越先前最佳系统。这种将 OS 经典内存管理技术引入 ML 系统的思路对未来系统设计具有启发意义。

译注 (v2 修正说明)

本翻译 (v2) 相比 v1 的主要修正 :

1. KV cache 大小公式 : 修正为原文准确表述 $2 \times \text{hidden_size} \times \text{num_layers} \times \text{bytes_per_element}$, 不再有多余的乘数
2. 三种内存浪费 : 严格按原文 3.1 定义为 $\text{Reserved} / \text{Internal fragmentation} / \text{External fragmentation}$ (不包含冗余复制——那是 4.3-4.4 讨论的共享问题)
3. 抢占策略 : 严格按原文 4.5 翻译为 Swapping (换出到 CPU) 和 Recomputation (丢弃重算)
4. 数据准确性 : vLLM 利用率 96.3% (非 98%) , Orca(Max) 为 20.4% (非 53%)
5. 关键段落附英文原文方便对照

原文链接 : <https://arxiv.org/abs/2309.06180>

vLLM 仓库 : <https://github.com/vllm-project/vllm>